

L3 总结:tool use 深入

日期:2026-07-14 结果:达标  (作业 refund_agent.py 三条测试跑通;quiz 3 满分 + 2 需补全)

这节课学到了什么(一句话)

在 L2 单工具 loop 的基础上,发现agent 的能力增量全在工具侧——loop 一行没改,靠加工具、打磨 description、处理错误,agent 就从「只会查」升级到「会退款、会链式多步、会从错误里恢复」。

核心内容

1. 关键认知:能力来自工具,不来自 loop

L2→L3 那个 while 循环结构几乎没动,能力却大涨。结论:loop 是通用固定引擎;想让 agent 会做新事,是加工具 + 写好 description,而不是改循环。这条贯穿后面所有课——L4 编排、L8 客服原型都不会再动 loop,只往工具和 prompt 上叠。(我们这个 loop 是 ReAct(Reason+Act,推理与行动交替)的简化版。)

2. 多工具:模型靠 description 选工具

工具越多,description 质量越生死攸关。写好三要点:说清做什么、何时用、何时别用(把业务规则前置)。模型决定「调不调、先查还是直接退」的唯一依据就是 description 里的文字——这是最便宜的非侵入式行为旋钮。

3. 错误也是信息(本节最重要的思想)

工具报错 ≠ agent 失败。用 try/except 接住异常,把错误文本作为 tool 结果回传给模型,模型会自愈:换参数、改调别的工具、或体面告知用户。– 坏:异常击穿 → Python 崩 → 用户看 Traceback。– 好:异常 → {"error": str(e)} 回传 → 模型解释原因、给方案。

4. 链式调用:loop 白送的超能力

「退掉 B456」→ 先 query_order 查状态 → 再 apply_refund。第二步依赖第一步,需要的新代码为零:第一轮工具结果 append 进 messages,第二轮模型读到「已签收」自己决定退款。多轮串行编排是涌现的,不是额外实现的。– 区分:并行(一轮点多单,互不依赖,如 L2 的 A123+B456)vs 链式/串行(跨轮,后依赖前)。

5. 兜底:循环次数上限

while True → while turn < MAX_TURNS,超限返回「转人工」话术而非烧 token。生产必备。(学习者在 L2 quiz Q4 自己提出,本节亲手实现。)

课堂上真实发生的事

作业一次跑通,三条测试行为全对

- B456(已签收):链式——先查后退,退款成功。
- A123(运输中):被业务规则拦下,模型不但说不能退,还自己补出「等签收再退 / 拒收」方案(工具结果里没有的自由发挥)。
- C999(不存在):错误回传 → 模型体面告知,程序不崩溃。schema(TODO1)与 try/except(TODO2)一次写对,description 措辞到位,模型乖乖「先查再退」。

两个真实工程教训(比 quiz 更值钱)

1. 兜底代码平时永远不跑,出事那天才跑——所以 TODO3 的「超限返回话术」最容易被漏写(学习者第一版只写了计数、漏了 return),也最难被正常测试发现。
2. 调试临时值要记得还原:测试兜底时把 `MAX_TURNS` 改成 0,忘了改回 8 → agent 彻底罢工(`0 < 0` 为假,循环一次不进,每次都掉到兜底话术)。防它:改前标注「临时」,commit 前 `git diff` 扫一眼。正好接住学习者已在用的 git 留痕习惯。

Quiz 结果

Q1 能力来自工具(不来自 loop)⚠️ 需补全 / Q2 tool 结果经 messages 回传给模型✅ / Q3 错误回传让模型自愈✅ / Q4 模型只看 description 决策✅ / Q5 30 工具的问题:选错(答到)+ schema 每轮吃 token 撑大上下文(需补全)。

掌握等级

Beginner 稳固、持续向 Intermediate 迁移。能独立扩展工具、处理工具错误、理解链式调用的涌现性,并抓住「能力来自工具」这一关键抽象。工程细节意识在增强(兜底、调试残留),仍需继续养成 commit 前查 diff 的习惯。

埋下的伏笔(后续呼应)

- 30 工具 → 选择困难 + schema 吃上下文 → L4 工具编排/分组、进阶「按需检索工具」(思路同 L7 RAG,检索对象从知识换成工具)。
- 「先查再退」的多步依赖 → L4 多工具编排 capstone(命令行多工具 agent)。
- 错误恢复、业务规则兜底 → L10 错误处理 / L8 客服原型 的安全设计。